

R5.04 - Qualité algorithmique

TP1 - Premiers programmes en C

2025-2026

La compilation, qu'est-ce que c'est ?

Contrairement à des langages interprétés comme Python, un programme en C doit au préalable être **compilé**, c'est-à-dire qu'à partir du **code source** nous allons générer un **code machine** stocké dans un **exécutable**. C'est ce code machine qui permettra d'indiquer au processeur les instructions à exécuter.

Bien sur, ce n'est pas vraiment nous qui allons générer l'exécutable, mais plutôt un programme appelé **compilateur**. Un compilateur très répandu est [gcc](#), nous l'utiliserons donc par la suite.

Attention



Par abus de langage, "compilation" désigne bien souvent tout le processus de transformation du code source en code machine. Cependant les choses sont un peu plus compliqué en [pratique](#).

Compilation avec gcc

Ouvrez un éditeur de texte et recopiez le programme suivant :

```
#include <stdio.h>

int
main(void)
{
    printf("Bonjour!\n");
    return 0;
}
```

Sauvegardez ce fichier que vous appellerez `bonjour.c`. Ouvrez ensuite un terminal, placez-vous dans le dossier contenant votre fichier nouvellement créé, puis exécutez la commande suivante :

```
$ gcc bonjour.c
```

Affichez maintenant le contenu de votre dossier. Un fichier `a.out` a normalement été produit! C'est votre exécutable. Pour l'exécuter, tapez la commande suivante :

```
$ ./a.out  
Bonjour!
```

Il est possible de renommer l'exécutable. Pour cela il faut rajouter l'option `-o <nom>`.

```
$ gcc bonjour.c -o bonjour
```

Il est également possible d'utiliser plusieurs fichiers pour générer notre exécutable. Créez un fichier `main.c` dans lequel vous copierez votre fonction `main`.

```
int  
main(void)  
{  
    bonjour();  
    return 0;  
}
```

Modifiez maintenant votre fichier `bonjour.c` afin qu'il contienne le code suivant.

```
#include <stdio.h>  
  
void  
bonjour()  
{  
    printf("Bonjour!\n");  
}
```

Compilez les deux fichiers avec la syntaxe suivante :

```
$ gcc main.c bonjour.c -o bonjour
```

Une erreur doit normalement apparaître... Que se passe-t-il selon vous?

Créez maintenant un fichier `bonjour.h` et ajoutez-y l'unique ligne suivante :

```
void bonjour();
```

Rajoutez ensuite la ligne `#include "bonjour.h"` au début du fichier `main.c` ainsi qu'à la deuxième ligne du fichier `bonjour.c`. Relancer la compilation. Normalement, les choses doivent mieux se passer!

Nous allons maintenant ajouter quelques options supplémentaires :

```
$ gcc -std=c11 -Wall -Wextra -O0 bonjour.c main.c -o bonjour
```

Voici quelques explication concernant les options proposées :

- `-std=c11` : permet de préciser le standard du langage C que nous souhaitons utiliser. En effet, la syntaxe et les fonctionnalités du langage évolue avec le temps. Ici nous avons choisi la [norme C11](#), qui inclut notamment, par rapport à la norme C99, la programmation multi-thread ainsi qu'une meilleur gestion de l'encodage UNICODE.
- `-Wall` : cette option permet d'afficher des avertissements sur un certain nombre d'erreurs de programmation. Votre programme pourra tout de même compiler mais la présence d'avertissement peut impliquer un comportement erroné lors de l'exécution de votre programme.
- `-Wextra` : d'autres avertissements, que nous détaillerons ultérieurement.
- `-O0` : désactive toute forme d'optimisation que votre compilateur pourrait utiliser.

Astuce

Il est également possible de compiler séparément les différents fichiers source puis de les lier. Pour cela, il faut utiliser l'option `-c` qui génère des fichiers objet avec l'extension `.o`. On peut ensuite passer à la phase d'édition de liens. Voici la marche à suivre

```
$ gcc -c bonjour.c
$ gcc -c main.c
$ gcc main.o bonjour.o -o bonjour
```



Compiler séparément les fichiers peut se révéler utile si l'on ne modifie qu'un seul fichier et que l'on veut s'épargner la recompilation inutile des autres.

Quelques détails sur les premières étapes de la compilation.

La génération de fichiers objet et la génération de l'exécutable (édition de liens) sont les dernières phases du processus de compilation. Nous allons nous attarder sur les étapes préliminaires.

Reprenons notre fichier `main.c` et commençons par exécuter la commande `gcc -E bonjour.c`. Faites la même chose avec le fichier `bonjour.c`. Qu'obtenez-vous?

Utilisons maintenant une nouvelle option. Dans le fichier `main.c`, commentez l'appel à la fonction `bonjour()` et exécutez la commande `gcc -s bonjour.c`. Vous devez obtenir un fichier `main.s`. Que contient ce fichier?

Les deux étapes que nous venons de mettre en évidence sont le preprocessing et les différentes étapes d'analyse (lexicale, syntaxique, sémantique), formant la partie frontale de la compilation.

Utilisation de make et premiers Makefiles

L'outil `make` permet d'automatiser le processus de compilation. Il est associé à un fichier appelé **Makefile**, qui décrit l'ensemble des **règles de compilation**.

Une règle se présente sous cette forme :

```
cible: dépendances
      commande
```

Par exemple, si on souhaite compiler notre fichier `bonjour.c`, la règle sera :

```
bonjour.o: bonjour.c bonjour.h
      gcc -Wall -c bonjour.c
```

On peut ainsi obtenir une première version de notre Makefile.

```
main: main.o bonjour.o
      gcc -Wall main.o bonjour.o -o main
main.o: main.c bonjour.h
      gcc -c main.c
bonjour.o: bonjour.c bonjour.h
      gcc -c bonjour.c
```

Cependant cette première version ne nous apporte pas grand chose par rapport à un simple script shell (excepté le fait qu'un fichier non modifié ne sera pas re-compilé).

Heureusement, `make` est bien plus puissant que ça. Nous avons notamment la possibilité d'utiliser des ***variables** ainsi que des **règles implicites**. Voici une deuxième version possible de notre Makefile.

```
CC=gcc
OBJ=main.o bonjour.o

main: $(OBJ)
main.o: main.c bonjour.h
bonjour.o: bonjour.c bonjour.h
```

```
.PHONY : clean
clean:
    rm -f main *.o
```

Le programme make va déduire de lui-même les commandes à passer au compilateur.

A vous de coder !

Récupérez sur Celene le programme droites.c qui contient un exemple d'utilisation de la bibliothèque SDL2. Cette bibliothèque permet de faire l'interface avec notamment votre carte graphique et est utilisée pour créer des jeux vidéo.

Ce premier programme consiste à dessiner un carré dans le plan et à effectuer une rotation. Pour le moment nous utilisons la primitive

```
SDL_RenderDrawLine(renderer, x1, y1, x2, y2)
```

Cette fonction permet de tracer un segment d'un point de coordonnées $(x1, y1)$ à un point de coordonnées $(x2, y2)$.

Le but du jeu est d'implémenter votre propre fonction de tracé en utilisant l'algorithme de Bresenham! Vous implémenterez la version naïve ainsi que la version plus optimisée.